

Reporte Entrega Final

MESH

UNIVERSIDAD DEL ISTMO

INGENIERÍA EN SISTEMAS Y CIENCIAS DE LA COMPUTACIÓN

Diseño y Administración de Bases de Datos 2

José Manuel González Corado

José David Navichoque Pineda

Juan Pablo Torres García

Guatemala, lunes 2 de noviembre del 2026

INDICE

1. INTRODUCCIÓN 3
1.1. Alcance de la entrega y repositorio 3
1.2. Cómo responde este informe a la rúbrica y a los lineamientos 3
1.3. Distribución de responsabilidades 4
2. RECOMENDACIÓN SEMÁNTICA DE PRODUCTOS 6
2.1. Por qué pgvector primero y su registro de decisión 6
2.2. Las revisiones, a Elasticsearch y de vuelta a pgvector 7
2.3. Cómo se calcula el vector de cada producto 8
2.4. La consulta de productos parecidos 8
2.5. Evidencia y dónde se ve 10
2.6. Búsqueda híbrida y por intención 11
3. CONTROL DE ACCESO 12
3.1. Un usuario por servicio y su registro de decisión 12
3.2. Los usuarios de Redis 13
3.3. Los usuarios de las demás bases 13
3.4. Puertos, herramientas e imágenes 14
3.5. Evidencia 15
3.6. Dentro de la tienda, por rol y por oferta 15
4. ARQUITECTURA FINAL 17
4.1. El diagrama de arquitectura 17
4.2. Qué guarda cada componente 19
4.3. Cómo se levanta el sistema completo 19
4.4. Qué pasa si solo quedan una, dos o tres bases 20
5. REGISTRO DE DECISIONES DEL SEMESTRE 22
5.1. Un solo registro 22
5.2. Las decisiones de septiembre que faltaban 24
5.3. Comprar sin MongoDB 26
5.4. Las decisiones de octubre 26
6. FRAUDE EN RESEÑAS Y CLASIFICADOR 31
6.1. Un solo modelo de señales 31
6.2. El clasificador de reseñas 31
7. RETROSPECTIVA 33
7.1. Cómo la hicimos 33
7.2. Lo que sostendríamos 33
7.3. Lo que replantearíamos 34
7.4. Lo que aprendimos sobre la forma de trabajar 35
8. CONCLUSIONES 36
8.1. Conclusiones 36

1. INTRODUCCIÓN

1.1. Alcance de la entrega y repositorio

Mesh es nuestra implementación de TiendaYa, una tienda en línea en la que varios vendedores comparten un mismo catálogo. Al llegar a esta entrega el sistema ya reparte sus datos entre seis motores. PostgreSQL es la fuente de verdad del dinero y del stock, MongoDB guarda el catálogo y su historial, Redis el carrito y la oferta limitada, Neo4j las relaciones entre cuentas y reseñas, ClickHouse los eventos en el tiempo y Elasticsearch el índice de búsqueda.

La Entrega final cierra la arquitectura con dos piezas. La recomendación de productos parecidos por el significado de su descripción, con vectores que empezaron en PostgreSQL con pgvector, pasaron unas semanas por Elasticsearch y volvieron a pgvector, y el control de acceso, que empezó en Redis con un usuario por servicio y desde el 23 de octubre está en las cinco bases que lo permiten. El informe incluye además el diagrama de arquitectura final, el registro único de las decisiones del semestre, lo que se sumó después con los mismos vectores y la retrospectiva.

El trabajo está en el repositorio `https://github.com/pinedahuella/Proyecto-Tienda-Base-Datos-2`, con 371 commits sobre `main` entre el 15 de agosto y el 6 de noviembre de 2026. Cada afirmación del informe sale de código que corre o de una prueba que se puede repetir, y cada prueba indica el comando que la reproduce. La primera versión de este informe es del 25 de septiembre, la segunda del 8 de octubre y la tercera del 20 de octubre. Esta versión suma el control de acceso en todas las bases, la seguridad de Elasticsearch, la foto en el buscador y un solo registro con las 26 decisiones del semestre. La última semana sumó dos ACL que viven en PostgreSQL, uno por rol para cada función y otro por oferta, y ordenó por categoría la recomendación de quienes lo compraron también llevaron. El 5 de noviembre los parecidos por el vector pasaron a ese mismo orden.

1.2. Cómo responde este informe a la rúbrica y a los lineamientos

Tabla 1. Cada criterio de la rúbrica de la Entrega final con la sección que lo cubre.

Criterio de la rúbrica	Puntos	Sección	Evidencia medida
Funcionalidad de recomendación semántica	5	2	1000 productos activos con vector en pgvector, búsqueda de vecinos con filtro y prueba con 15 comprobaciones en verde

Criterio de la rúbrica	Puntos	Sección	Evidencia medida
Control de acceso configurado en un componente NoSQL	4	3	Un usuario por servicio en Redis, MongoDB, ClickHouse, Elasticsearch y PostgreSQL. La prueba de las cinco bases dio 50 comprobaciones en verde y la de Redis 43, cada negativa con su control. Dentro de la tienda, un ACL de funcionalidades por rol y otro por oferta, en PostgreSQL, que contestan 403 aunque se llame a la ruta directo
Diagrama de arquitectura final, completo y correcto	3	4	Diagrama generado con `docs/generar-arquitectura.py` con los seis motores, Ollama, el usuario con que entra cada servicio y las bases en otra PC
Documento de retrospectiva	4	7	Siete decisiones que sostendríamos y ocho que replantearíamos, cada una con lo que pasó de verdad y su registro
Presentación del proyecto	4	fuera del informe	Las diapositivas en `presentacion-final/index.html`, las demos del panel de comandos y `PREGUNTAS-SUSTENTACION.md`

Tabla 2. Los lineamientos generales del enunciado y dónde se cumplen.

Lineamiento del enunciado	Dónde se cumple
5.1 Distribución de responsabilidades	Sección 1.3
5.2 Componentes de referencia	Empezó con la extensión vectorial sobre PostgreSQL, la opción de referencia, pasó por `dense_vector` de Elasticsearch y volvió a pgvector el 16 de octubre
5.3 Registro de decisiones arquitectónicas	Sección 5, un solo registro de ADR-1 a ADR-26 con los cinco campos. ADR-13 en 2.1 y sus dos revisiones en 2.2, y ADR-14 en 3.1
5.4 Documentación para levantar el sistema	Sección 4.3 y `docs/instalacion.md`, un solo comando que también crea los usuarios de cada base y calcula los vectores

1.3. Distribución de responsabilidades

Mantuvimos el reparto de todo el semestre. José David Navichoque Pineda se encargó del backend, José Manuel González Corado del frontend, las migraciones, los ETL, los scripts de cada base, las pruebas, la documentación y el arranque, y Juan Pablo Torres García de los módulos de ClickHouse y Neo4j. Lo que una parte necesitaba de la otra quedaba escrito en `docs/BACKEND-PENDIENTE.md`, con la consulta ya probada.

Tabla 3. Quién hizo cada parte de la Entrega final.

Componente	José David Navichoque, backend	José Manuel González, datos y frontend	Juan Pablo Torres, ClickHouse y Neo4j
Recomendación semántica	La ruta de parecidos, el worker de vectores y las dos mudanzas del vector	Carga de vectores con Ollama, migraciones 024, 027, 036 y 040, prueba y estante de la ficha	

Componente	José David Navichoque, backend	José Manuel González, datos y frontend	Juan Pablo Torres, ClickHouse y Neo4j
Control de acceso	Revisión de los comandos que el backend usa en cada base	Usuarios por servicio en las cinco bases, contraseñas automáticas, Elasticsearch con seguridad y prueba de acceso	Los usuarios de ClickHouse y la revisión de roles en Neo4j
Eventos y tendencias	Las rutas de tendencias y los rankings	El ETL de compras y la carga de eventos	Los módulos de eventos, tendencias y combos en ClickHouse
Grafo	Las rutas de recomendaciones y de anillos	Los ETL y las colas del grafo	Los módulos de anillos de reseñas y de compras en Neo4j
Fraude en reseñas	Los detectores de reseñas repetidas, parecidas, con nota que no cuadra y canceladas	Migraciones 031 a 039, panel de Fraude y demos	
Arquitectura y retrospectiva		Diagramas, registro de decisiones, retrospectiva, informe y presentación	

Tabla 4. Commits por integrante en el historial de git al 6 de noviembre de 2026.

Integrante	Usuario en git	Commits
José Manuel González Corado	José Manuel González Corado	248
José David Navichoque Pineda	pinedahuella	118
Juan Pablo Torres García	zapatoazul	5
Total		371

2. RECOMENDACIÓN SEMÁNTICA DE PRODUCTOS

2.1. Por qué pgvector primero y su registro de decisión

Hasta esta entrega la tienda solo sabía relacionar productos por su categoría o por lo que otros compradores miraron. Dos productos que sirven para lo mismo pero están descritos con otras palabras no se encontraban. El enunciado pide sugerencias basadas en la similitud semántica de la descripción, con representaciones vectoriales.

Tabla 5. Registro de decisión ADR-13

ADR-13 Dónde guardar y consultar los vectores de los productos	
Contexto	Cada producto necesita un vector que represente el significado de su texto y una consulta que encuentre los más cercanos a uno dado. El catálogo tenía entonces 3105 productos activos y la consulta se hace una vez por cada ficha que se abre.
Alternativas consideradas	No introducir cambios y recomendar por categoría y precio parecido, que ya se podía con MongoDB y ClickHouse. Un componente vectorial dedicado, como Qdrant o Milvus. El tipo `dense_vector` de Elasticsearch, que ya corre en el sistema. La extensión pgvector sobre el PostgreSQL existente.
Decisión	pgvector sobre el PostgreSQL existente, con una tabla `producto_vector`, un índice HNSW y la función `productos_similares()`.
Justificación	Por patrón de consulta, buscar los vecinos de un producto entre unos miles es justo lo que un índice HNSW resuelve en milisegundos. Por costo operativo, la imagen `pgvector/pgvector:pg18` venía en el sistema desde la Entrega 1, así que no hubo contenedor nuevo ni otra copia de los identificadores. Por consistencia, el vector vive junto a las ofertas y los pedidos y se puede cruzar con ellos en la misma consulta. Elasticsearch también servía, pero su índice se reconstruye entero cuando cambia el mapeo y los vectores tardan minutos en calcularse. La primera alternativa quedó descartada porque filtrar por campos no mide el significado del texto.
Consecuencias	Ganamos la recomendación sin operar un motor más. Aceptamos que el vector es una copia derivada del texto de MongoDB y hay que recalcularlo cuando ese texto cambia, lo que el ETL resuelve comparando un hash. También que PostgreSQL recibe una consulta más por ficha y que calcular los vectores necesita un modelo local. A millones de productos la decisión se revisaría.

Esta decisión se sostuvo hasta el 4 de octubre. La siguiente sección explica las dos revisiones, la que llevó el vector a Elasticsearch y la que lo devolvió a pgvector.

2.2. Las revisiones, a Elasticsearch y de vuelta a pgvector

Tabla 6. Registro de decisión ADR-13, primera revisión

ADR-13, primera revisión Dónde guardar y consultar los vectores, primera revisión	
Contexto	Con pgvector, buscar parecidos que estuvieran activos y no fueran copias pedía una tabla aparte con su propia copia de los identificadores y del estado. El buscador de texto vivía en otro motor, y la búsqueda híbrida necesitaba el vector junto al texto y las facetas. Las reseñas también iban a tener vector, para buscar por intención y para detectar textos casi iguales.
Alternativas consideradas	Dejar pgvector como fuente y copiar el vector a Elasticsearch solo para la búsqueda. Mudar el vector a Elasticsearch. No cambiar nada y no hacer la búsqueda híbrida.
Decisión	Mudar el vector a Elasticsearch. Cada producto lleva el suyo en su documento de `mesh_productos` y cada reseña en el índice nuevo `mesh_comentarios`. La migración 036 retira `producto_vector` y `productos_similares()`.
Justificación	En Elasticsearch el vector vive en el mismo documento que el nombre, el estado y las facetas, así que una sola consulta los combina y no hay una segunda copia que se pueda quedar atrás. Copiarlo desde pgvector dejaba dos lugares con el mismo dato, que es justo lo que la retrospectiva marca como el error que más se repitió. El tipo `dense_vector` y la búsqueda de vecinos vienen en la licencia básica.
Consecuencias	Se pierde poder cruzar el vector con las compras en una consulta SQL, que hoy nadie usa. Rehacer el índice ya no pierde los vectores, porque antes se guardan en `mesh_productos_vectores_previos` y vuelven sin pasar por Ollama, en 48 segundos y no en 7 minutos. La imagen de PostgreSQL con pgvector se queda, porque cambiarla toca el volumen de datos.

Esa mudanza duró hasta el 16 de octubre. En la revisión de ese día el profesor pidió que las consultas por vectores se hicieran con pgvector, y al medir las dos opciones contra el coseno exacto salió otro argumento a favor de volver.

Tabla 7. Registro de decisión ADR-13, segunda revisión

ADR-13, segunda revisión Los vectores vuelven a PostgreSQL con pgvector	
Contexto	Las consultas por vectores tenían que hacerse con pgvector. Antes de mudar se midió la precisión de cada motor. En 1001 productos, los parecidos de Elasticsearch coincidían con el coseno exacto en 317, porque su índice `int8_hnsw` guarda los vectores comprimidos y se equivoca hasta en 0,016.
Alternativas consideradas	Dejar el vector en Elasticsearch y copiarlo a PostgreSQL solo para las consultas. Devolver el vector a PostgreSQL y dejar en Elasticsearch solo el texto. No cambiar nada.
Decisión	Devolver todos los vectores a PostgreSQL. La migración 040 crea `vector_producto` y `vector_comentario`, una fila por producto o por reseña, con llave foránea, borrado en cascada y un índice HNSW por coseno. Las cuatro búsquedas que usan vectores le piden los vecinos a PostgreSQL y Elasticsearch se queda con el texto.
Justificación	pgvector coincidió con el coseno exacto en 1000 de 1001 productos, y el otro era un empate. Con una sola copia del vector no hay dos lugares que se puedan quedar atrás. La extensión venía en la imagen de PostgreSQL desde la Entrega 1, así que no hubo contenedor nuevo. Las tablas van aparte de `comentario` porque un vector pesa unos 3 KB y haría más pesada cada lectura de reseñas.
Consecuencias	La búsqueda híbrida pasa a ser dos consultas, las palabras en Elasticsearch y el significado en PostgreSQL, que el backend junta por posición. Si Elasticsearch se cae, los parecidos de la ficha siguen saliendo. Los índices `mesh_comentarios` y `mesh_productos_vectores_previos` se retiraron, y el arranque copia los vectores que estaban en Elasticsearch sin volver a pedirlos a Ollama.

2.3. Cómo se calcula el vector de cada producto

El texto de cada producto se arma con su nombre, su categoría y su descripción, leídos de MongoDB. Ese texto se convierte en un vector de 768 números con el modelo `paraphrase-multilingual`, que entiende español y está entrenado para que dos frases con el mismo sentido den vectores cercanos aunque usen palabras distintas.

El modelo corre en Ollama, en la misma máquina. Ningún texto sale a un servicio externo y no hay costo por consulta. El arranque usa el Ollama de la PC si ya lo tiene y, si no, levanta el contenedor `mesh-ollama` y baja el modelo solo, así que nadie del equipo tuvo que instalar nada a mano.

- `etl/cargar_vectores.ps1` guarda en `vector_producto` el md5 del texto con que se calculó cada vector, en `texto_hash`. En un arranque normal solo recalcula los productos cuyo texto cambió y tarda unos segundos.
- Entre un arranque y otro, el backend revisa cada 10 segundos los productos que un vendedor creó o editó, con el mismo texto y el mismo md5. Si solo cambiaron las variantes o el estado, no le pide nada a Ollama.
- La primera carga, con el catálogo de 3105 productos de entonces, tardó 413 segundos.
- El generador de datos de prueba copia productos con otro sufijo en el nombre y la misma descripción. Sin cuidarlo, los seis parecidos de un teléfono eran seis copias del mismo teléfono. La columna `grupo` guarda el md5 de la categoría y la descripción sin el nombre, y la consulta devuelve un solo producto por grupo.

2.4. La consulta de productos parecidos

El backend lee el vector del producto y le pide a PostgreSQL sus vecinos más cercanos por coseno, con el índice HNSW. El filtro va en la misma consulta, solo productos activos, sin el propio producto y sin las copias de su grupo. Después se queda con el mejor de cada grupo y corta en seis.

```
SELECT id_producto_mongo, grupo,
       1 - (vector <=> $1::vector) AS similitud
  FROM vector_producto
 WHERE activo
   AND id_producto_mongo <> $2
   AND grupo <> $3
 ORDER BY vector <=> $1::vector
 LIMIT 200
```

El núcleo de la búsqueda de parecidos en backend/src/services/productos/similares.service.ts.

El operador `<=>` de pgvector mide la distancia coseno, de 0 a 2, así que el parecido es uno menos la distancia, en la escala de 0 a 1 que enseña la ficha. El vector va como parámetro y no como subconsulta, porque con una subconsulta el planificador no usa el índice HNSW.

Desde el 5 de noviembre el estante ordena por bloques, con el mismo esquema que quienes lo compraron también llevaron (ADR-25). Antes de la consulta de arriba corre otra igual pero solo sobre los productos activos de su misma categoría, que salen de `producto_catalogo`. Los seis se llenan en ese orden, primero todos los de su categoría, del más parecido al menos, y lo que falte con el resto del catálogo, siempre uno por grupo. Un producto de la misma categoría con 0,40 de parecido va antes que uno de otra con 0,80, y cada resultado dice si es de la misma categoría en `misma_categoria`. La consulta del resto sigue sin JOIN para que use el índice HNSW.

Tabla 8. Los seis parecidos del producto 70 el 5 de noviembre. La cómoda se parece más que la bomba de piso, pero es de otra categoría y va al final.

Maillot de ruta Peloton, Ciclismo	Parecido	Bloque
Candado plegable Cerrojo F8	0,707	Su categoría
Guantes de ciclismo Palma Corta L1-0728	0,639	Su categoría
Casco de ruta Aerodino	0,624	Su categoría
Sillin de ruta Vertebra SR-2 L1-0822	0,549	Su categoría
Bomba de piso Piston	0,456	Su categoría
Comoda Sendero 4 gavetas L1-0779	0,657	El resto

Con pgvector aprendimos a revisar si el índice de vecinos se usa de verdad. La primera versión de la función nunca lo usaba, y la migración 027 lo corrigió, de 32 a 3,8 ms por producto. La misma revisión encontró un problema con las reseñas, que entonces estaban en Elasticsearch. Con textos repetidos muchas veces, el grafo de vecinos se atascaba y no encontraba las mejores. Al volver a pgvector apareció lo mismo con otra cara. Con los valores por defecto el índice HNSW revisa 40 candidatos, el filtro de activos descarta algunos y un `LIMIT 200` volvía con 39 filas. Por eso cada búsqueda sube `hnsw.ef_search` y activa `hnsw.iterative_scan` solo dentro de su transacción, y con eso vuelven las 200.

Tabla 9. Medición del 4 de octubre en el índice de reseñas de Elasticsearch. Por eso aquel índice comparaba contra todas.

Búsqueda para zapatos cómodos para caminar todo el día	Mejor reseña encontrada
Grafo de vecinos con 100 candidatos	0,30, una reseña repetida que no habla de zapatos
Grafo de vecinos con 400 candidatos	0,81
Comparar contra todas	0,81

2.5. Evidencia y dónde se ve

La prueba `python pruebas/recomendacion_semantica.py` corre contra las bases en funcionamiento y dio 21 comprobaciones en verde el 5 de noviembre, con el vector en PostgreSQL. Revisa que cada producto de MongoDB esté en la copia del catálogo y tenga su vector de 768 dimensiones, todos del mismo modelo, que solo por el vector, calculado contra todos en PostgreSQL, los vecinos compartan categoría mucho más seguido de lo que daría el azar, 66 por ciento contra 6 en la muestra de esa corrida, con ese azar medido sobre los mismos productos como control, que la ficha ponga primero todos los de su categoría y que `misma_categoria` diga la verdad, que la búsqueda no devuelva el propio producto, productos inactivos ni copias del mismo producto, y que un producto sin vector devuelva una lista vacía sin romper nada. La parte de los bloques se probó también con el orden anterior y falló, como debía. `node pruebas/vectores_worker.cjs` comprueba con 5 en verde que el worker del backend calcula el mismo vector que el ETL, con distancia cero.

Al modelo nunca se le dice la categoría como dato aparte, solo la lee dentro del texto. Que los vecinos caigan en la misma categoría es la señal de que el vector captura de qué trata el producto, y por eso esa cifra se mide solo con el vector y no con el orden por bloques. Medido en los 1000 productos activos el 5 de noviembre, solo por el vector el 68 por ciento de los vecinos es de su categoría, y en la ficha, con su categoría primero, el 84,5 por ciento. La ruta dio los mismos seis, en el mismo orden y con los mismos bloques, que el cálculo exacto contra todos en los 1000.

En la tienda, la ficha de cada producto tiene el estante Parecidos a este, que pide los parecidos a la ruta `GET /productos/:id/similares` del backend. Para enseñarlo en clase están `.\demo-entrega-final.ps1 -Parte semantica` y `-Parte vectores`, que buscan un producto por nombre, muestran sus seis parecidos con el porcentaje de similitud, separados en sus dos bloques, y comprueban que la ruta devuelve los mismos productos en el mismo orden.

2.6. Búsqueda híbrida y por intención

Con los vectores, el buscador sumó dos formas de buscar. La híbrida corre la búsqueda por palabras en Elasticsearch y la búsqueda por significado de la frase en PostgreSQL, y mezcla las dos listas por posición con la fusión por rango recíproco, con la constante habitual de 60. Así una frase puede encontrar un producto aunque no comparta ninguna palabra con su nombre.

La búsqueda por intención compara la frase con el vector de los productos y también con el de las reseñas, y enseña lo que cuentan otros compradores. Los dos usan el mismo modelo de Ollama que la recomendación, así que la frase de la persona se convierte en vector en la misma máquina.

- La página de resultados solo enseña un producto con similitud de 0,6 o más y una reseña con 0,5 o más. Por debajo, la búsqueda por significado siempre devolvería algo, aunque no tenga relación.
- La híbrida todavía no agrupa las copias del generador de datos como sí lo hace la recomendación, y eso queda como mejora.
- Al proponer un producto, el vendedor recibe sugerencias de categoría y de productos parecidos con el mismo vector.
- Se enseñan con las partes `hibrida`, `intencion` y `sugerencias` de `.\demo-entrega-final.ps1`.

3. CONTROL DE ACCESO

3.1. Un usuario por servicio y su registro de decisión

El enunciado pide control de acceso en al menos un componente NoSQL. Lo hicimos primero en Redis, el 25 de septiembre, y el 23 de octubre lo llevamos a todas las bases que tienen roles, porque revisando la seguridad encontramos que el backend entraba a MongoDB como root y que Elasticsearch no pedía contraseña aunque su puerto se abre a la red para el modo remoto.

Tabla 10. Registro de decisión ADR-14

ADR-14 Control de acceso por servicio en cada base	
Contexto	Hasta el 25 de septiembre Redis tenía una sola contraseña, y hasta el 23 de octubre las demás bases entraban con su usuario de arranque. El backend entraba a PostgreSQL y a ClickHouse como dueño, a MongoDB como root y a Elasticsearch sin contraseña. DbGate, la ventana de MongoDB, no pide login y entraba como root. Las bases corren en otra PC y se usan por Tailscale, así que sus puertos están abiertos a la red.
Alternativas consideradas	1. Dejar un usuario de arranque por base. 2. Control de acceso solo en Redis, lo mínimo del enunciado. 3. Un usuario por servicio en cada base que tenga roles. 4. Un proxy delante de las bases que filtre lo que se puede hacer.
Decisión	Un usuario por servicio. En Redis cuatro usuarios de ACL desde el 25 de septiembre, con el del backend limitado a las claves `mesh:*` desde el 23 de octubre. En PostgreSQL, MongoDB, ClickHouse y Elasticsearch un usuario `mesh_backend` que solo lee y escribe datos y un `mesh_lectura` que solo mira, desde el 23 de octubre. Elasticsearch con la seguridad encendida. Neo4j queda con un usuario porque la edición community no trae roles.
Justificación	**Patrón de consulta.** El backend solo lee y escribe datos. Antes de recortar se revisó su código y ninguna ruta crea tablas, colecciones o índices, así que su usuario puede quedarse sin esos permisos sin perder nada. **Consistencia.** Los permisos están escritos en el repositorio, en `seguridad/` y en `docker-compose.yml`, y el arranque los vuelve a aplicar cada vez. Una base que alguien cambió a mano vuelve a quedar igual. **Costo operativo.** Las contraseñas nuevas las genera el arranque si faltan y las pone en el `.env` del backend, nadie las teclea. El modo remoto toma las de la otra PC y, si esa PC todavía no tiene los usuarios nuevos, usa el de antes para no dejar al backend sin arrancar. **Escalabilidad.** Agregar un servicio es agregar un usuario en un archivo. El proxy de la alternativa 4 resolvía lo mismo con un componente más y sin los roles que cada base ya trae.
Consecuencias	Un error o una inyección en el backend ya no puede borrar tablas, colecciones ni índices, ni leer bases que no son suyas, y Elasticsearch ya no se puede leer ni borrar sin contraseña. Aceptamos más contraseñas en el `.env`, que la PC remota corra el arranque nuevo antes de usar los usuarios del backend y que Neo4j siga con un solo usuario. El dueño de cada base queda para las migraciones, el ETL y las pruebas.

La primera versión de este registro decidía solo Redis y lo justificaba con que Redis guardaba lo que no se puede reconstruir, los carritos abiertos y el stock de una venta relámpago. Desde el 17 de octubre eso ya no es cierto. La verdad del carrito y de las reservas está en PostgreSQL y Redis guarda una copia rápida (ADR-21), y por eso esa justificación salió del registro.

3.2. Los usuarios de Redis

Tabla 11. Los cuatro usuarios de ACL de Redis y sus permisos.

Usuario	Quién lo usa	Puede	No puede
`backend`	El backend	Leer y escribir las claves `mesh:*`, poner vencimientos, correr scripts Lua sobre ellas y pedir INFO	Tocar claves fuera de `mesh:*` y todo lo de la categoría peligrosa, como FLUSHALL, CONFIG, KEYS, ACL y SHUTDOWN
`default`	Los scripts y las pruebas	Lo mismo que `backend` sobre cualquier clave	Todo lo de la categoría peligrosa
`inspeccion`	RedisInsight	Leer claves, ver su tiempo de vida, SCAN e INFO	Escribir, borrar, cambiar vencimientos o correr scripts
`operador`	Mantenimiento a mano	Todo	

Todas las claves de la tienda empiezan con `mesh:`, así que al usuario del backend se le dio solo ese prefijo. Un script Lua tampoco puede salir de él, porque Redis revisa los permisos de cada comando dentro del script. A la categoría peligrosa se le devolvió solo INFO, porque la biblioteca del backend lo pide al conectarse.

El comando de Redis va escrito como lista en `docker-compose.yml`. Escrito en una sola línea, Compose cortaba la contraseña en el signo que la precede y Redis arrancaba con una contraseña distinta a la del `.env`.

3.3. Los usuarios de las demás bases

Tabla 12. Los usuarios de PostgreSQL, MongoDB, ClickHouse, Elasticsearch y Neo4j.

Base	mesh_backend, el backend	mesh_lectura, para mirar	Dónde está
PostgreSQL	Lee, agrega, cambia y borra filas, usa las secuencias y las funciones. No crea, cambia ni vacía tablas ni crea usuarios	Solo SELECT, y su sesión es de solo lectura	`seguridad/postgres.sql`
MongoDB	Un rol hecho a medida, `mesh_backend_rol`, que busca, inserta, cambia y borra documentos de la base de la tienda. No crea ni borra colecciones ni índices	El rol `read` sobre la base de la tienda. Es el usuario de DbGate	`seguridad/mongo.js`
ClickHouse	`mesh_backend_rol`, lee y agrega eventos. No borra, no cambia tablas y no ve los usuarios	`mesh_lectura_rol`, solo lee	`seguridad/clickhouse.sql`

Base	mesh_backend, el backend	mesh_lectura, para mirar	Dónde está
Elasticsearch	`mesh_backend_rol`, lee y escribe documentos en los índices `mesh_*` y mira la salud. No crea ni borra índices ni administra usuarios	`mesh_lectura_rol`, lee `mesh_*` y en Kibana solo mira	`seguridad/elastic.sh`
Neo4j	El único usuario, `neo4j`	No existe	`docker-compose.yml`

En MongoDB no se usó el rol de fábrica `readWrite`, porque también deja borrar colecciones y crear índices, y el backend nunca lo hace. En Elasticsearch los índices los crea el arranque con el superusuario `elastic`, y Kibana habla con Elasticsearch con su propio usuario de sistema, `kibana_system`. Las contraseñas las crea el contenedor `elastic-usuarios`, que corre una vez en cada arranque igual que el de las vistas de Kibana.

Neo4j es la única base sin un usuario por servicio. En la edición community no existen los roles, y crear uno, darle permisos o negárselos son órdenes de la edición enterprise, que es de pago. Lo comprobamos en vivo y la prueba de acceso guarda lo que contesta Neo4j al intentarlo. Lo que sí se hizo fue lo que queda a mano, contraseña obligatoria, el navegador del puerto 7474 solo en la misma PC y el puerto de bolt como única puerta.

3.4. Puertos, herramientas e imágenes

Al revisar la seguridad encontramos tres cosas más que corregimos el mismo día. Elasticsearch tenía la seguridad apagada con un comentario que decía que solo escuchaba en la misma PC, y no era cierto, su puerto se abría a toda la red. Las herramientas de inspección, pgAdmin, DbGate, RedisInsight y el navegador de Neo4j, también se abrían a la red, y DbGate entraba a MongoDB como root sin pedir login. Y cuatro imágenes usaban la etiqueta `latest`, así que dos PC del equipo podían correr versiones distintas sin saberlo.

- Las bases siguen escuchando hacia la red porque el modo remoto las usa desde otra PC por Tailscale, que va cifrado, y las seis piden usuario y contraseña.
- Las herramientas y el puerto nativo de ClickHouse solo escuchan en `127.0.0.1`, se abren desde la misma PC y nadie de la red las alcanza.
- Todas las imágenes llevan versión fija, pgAdmin 9.17, DbGate 7.2.5, RedisInsight 3.8.0 y Ollama 0.40.5, además de las que ya la tenían.

3.5. Evidencia

La prueba de las cinco bases es `bash seguridad/prueba_acceso.sh` y dio 50 comprobaciones en verde, más las 43 de `redis/prueba_acceso.sh`, que corre al final. Cada negativa tiene su control. Que `mesh_lectura` no pueda escribir solo cuenta si `mesh_backend` sí pudo, y que `mesh_backend` no pueda crear una tabla solo cuenta si el dueño sí pudo. Sin esos controles, una base caída haría pasar todas las negativas.

Nada de lo que la prueba intenta puede dañar la tienda si la configuración estuviera mal. Lo que borraría algo se intenta dentro de una transacción que se deshace, sobre un objeto que no existe, o se le pregunta antes a la base si lo dejaría pasar, con `ACL DRYRUN` en Redis y con la consulta de privilegios en Elasticsearch.

Tabla 13. Lo que comprueban seguridad/prueba_acceso.sh y redis/prueba_acceso.sh.

Base	Qué se comprueba
PostgreSQL	Cada usuario entra con su contraseña y no con otra. `mesh_backend` no es superusuario, lee y cambia filas, y no crea, borra ni vacía tablas ni crea usuarios. `mesh_lectura` lee y no cambia nada
MongoDB	`mesh_backend` cuenta y cambia productos, y no crea índices, no borra colecciones, no lee los usuarios ni crea otros. `mesh_lectura` lee y no cambia
ClickHouse	`mesh_backend` lee y agrega eventos, y no ve los usuarios, no borra eventos ni crea o borra tablas. `mesh_lectura` lee y no agrega. El usuario de fábrica no entra sin contraseña
Elasticsearch	Sin usuario o con otra contraseña no se entra. `mesh_backend` busca y escribe en `mesh_productos`, y no crea ni borra índices, no lee los de Kibana ni administra usuarios. `mesh_lectura` busca y no escribe
Neo4j	Entra con su contraseña, no con otra, y se niega a crear un rol por ser la edición community
Redis	Los cuatro usuarios, con `backend` encerrado en `mesh:*`

En clase se enseña con `.\demo-entrega-final.ps1 -Parte acceso`, que pone a cada usuario a intentar lo mismo en Redis y muestra lo que contesta la base.

3.6. Dentro de la tienda, por rol y por oferta

Lo de arriba es por servicio, y lo hacen cumplir las bases. Ninguna base sabe qué persona está detrás de una petición, porque todas llegan como `mesh_backend`, así que lo que puede hacer cada cuenta dentro de la tienda lo decide la aplicación. El profesor pidió que el control de acceso esté también a nivel de funcionalidades y que se vea llamando al backend directo, sin la página, con una cuenta que no tiene permiso. Por eso hay tres capas, y las dos de adentro son listas en PostgreSQL.

Tabla 14. Las tres capas del control de acceso de Mesh.

Capa	Qué pregunta	Dónde vive	Si no
Las bases, por servicio	Qué puede hacer el backend o una herramienta en cada base	Usuarios y roles de cada base (ADR-14)	La base rechaza el comando
Las funciones, por rol	Qué acciones de la tienda puede usar cada rol	`rol_permiso`, migración 050 (ADR-26)	403 con el permiso que falta
Cada oferta, por persona	Qué cuenta puede ver, editar o descontar una oferta ajena	`permiso_oferta`, migración 049 (ADR-24)	404 sin ninguna entrada, 403 sin esa

Cada una de las 136 rutas privadas declara la acción que hace, por ejemplo `ofertas.publicar`, y un guard del backend le pregunta a `rol_permiso` en ese momento si el rol de la cuenta la tiene. Ese rol se lee de la base y no del token con que la cuenta entró, así que también un cambio de rol vale en la petición siguiente. Antes esa lista estaba escrita en el código. Ahora son datos, y quitar una fila vale en la petición siguiente, sin compilar ni reiniciar. Cada intento negado queda en `auditoria_permiso` con quién, su rol y la ruta que llamó, y el administrador ve y cambia la matriz en el panel.

Antes de cambiarlo se guardó qué roles entraban a cada una de las 165 rutas, leído del backend compilado, y `pruebas/permisos_cobertura.cjs` comprueba que con la tabla nueva entran exactamente los mismos. `pruebas/permisos_rol.sh` llama a las rutas directo con los tres roles y los 38 permisos, quita y devuelve permisos en vivo y le cambia el rol a una cuenta con la sesión abierta, con 171 comprobaciones en verde. En clase se enseña con `.\demo-entrega-final.ps1 -Parte permisos`.

Para la presentación el profesor pidió dos pruebas sin script. La primera es entrar en vivo a cada base con un usuario que no tiene el permiso y ver que la base lo rechaza, con las órdenes de `docs/GUIA-ESTUDIO-ACL.md`. La segunda es llamar las rutas directo con Postman. La colección de `presentacion-final/postman` entra como compradora, como vendedor y como administrador, pide lo que cada uno no tiene y espera 401 o 403, comparte una oferta y la devuelve, y llama a Elasticsearch y ClickHouse con los usuarios de servicio. Una última carpeta intenta una inyección SQL en el inicio de sesión, en el id de un pedido y en el buscador, y no pasa, porque cada consulta manda lo escrito aparte, como un valor, y los id se revisan antes de llegar a la base. Cada petición lleva su prueba y su control, y el 6 de noviembre las 64 dieron lo esperado, 56 de permisos y 8 de inyección, con Newman y dentro de Postman. En la misma carpeta, `importar-a-postman.ps1` mete la colección y su ambiente en el Postman de quien lo corra y toma las claves del `.env`, así que ninguna clave pasa por git.

4. ARQUITECTURA FINAL

4.1. El diagrama de arquitectura

Las flechas continuas son lo que hace el backend con cada base, y las punteadas y numeradas son datos que viajan de una base a otra. El flujo 11 lleva el texto de cada producto de MongoDB a Ollama, que tiene su propia caja, y guarda su vector en PostgreSQL. Las reseñas no llevan flecha, porque su vector va de la cola `evento_comentario` a `vector_comentario` dentro de la misma base. Cada caja dice con qué usuario entra el backend, y el recuadro de las bases dice que pueden correr en otra PC y usarse por Tailscale. Los dos ACL de la tienda, por rol y por oferta, son tablas de PostgreSQL y no llevan flecha. El diagrama está al 2 de noviembre.

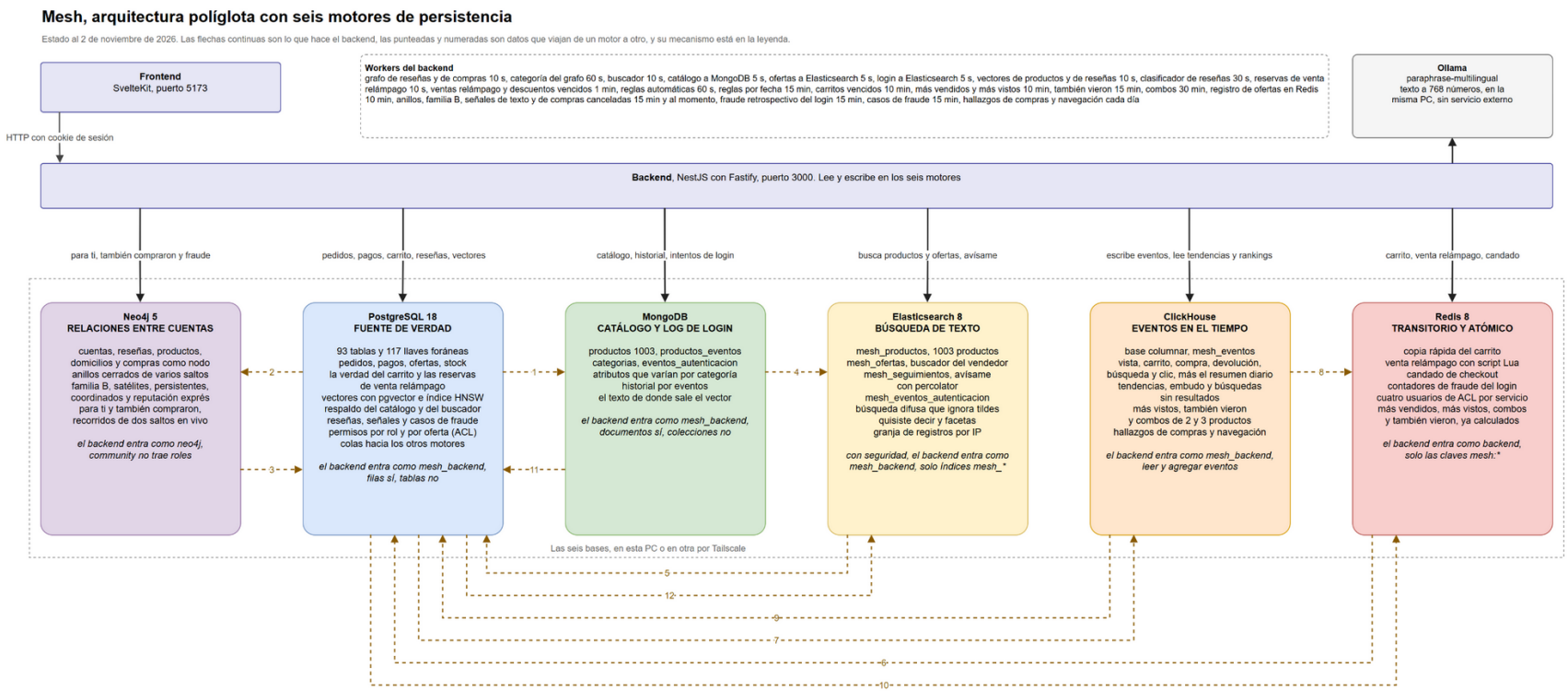


Figura 1. Arquitectura final de Mesh al 2 de noviembre de 2026, generada con docs/generar-arquitectura.py.

4.2. Qué guarda cada componente

Tabla 15. Los componentes de persistencia de Mesh y su rol.

Componente	Qué guarda	Desde	Por qué ahí
PostgreSQL 18	Cuentas, ofertas, stock, pedidos, pagos, la verdad del carrito, reseñas, reglas de descuento, señales y casos de fraude y la copia del catálogo que usa el checkout	Fase inicial	El dinero y el stock necesitan una transacción atómica
MongoDB	Catálogo con atributos por categoría, historial de eventos del producto y registro de inicios de sesión	Entrega 1	Cada categoría trae atributos distintos y la ficha se lee entera
Redis	Copia rápida del carrito, venta relámpago, candado del checkout, contadores de fraude y listas calculadas	Entrega 2	Datos de paso con tiempo de vida y operaciones atómicas bajo concurrencia
Neo4j	Cuentas, reseñas, compras y productos como grafo	Entrega 2	Los anillos de reseñas y las recomendaciones por compras son recorridos de varios saltos
ClickHouse	Eventos de vista, carrito, búsqueda, compra y devolución	Entrega 2	Preguntas nuevas por intervalos de tiempo sobre muchos eventos
Elasticsearch	Índice de búsqueda de productos con la foto de cada uno, buscador de ofertas, consultas guardadas de los avisos y copia del registro de inicios de sesión	Entrega 3	Tolerancia a errores, relevancia y facetas en una sola consulta
pgvector	Los vectores de productos y reseñas, del 25 de septiembre al 4 de octubre y otra vez desde el 16 de octubre	Entrega final	Coincide con el coseno exacto, ver 2.2

El diagrama entidad relación se genera leyendo el esquema de la base en funcionamiento con `docs/generar-er.py`, y el script falla si una tabla o una llave foránea queda sin dibujar, así que no puede quedar atrás de las migraciones. Hoy son 93 tablas y 117 llaves foráneas, de las migraciones 001 a 050.

4.3. Cómo se levanta el sistema completo

Todo se levanta con `.\arrancar.ps1`. El script prende los contenedores, corre las migraciones, carga los datos, reconstruye el catálogo en MongoDB, el índice de búsqueda y el grafo, pone al día la copia del catálogo en PostgreSQL, calcula los vectores que falten, crea los usuarios de cada base y al final cuenta lo que quedó en cada base contra PostgreSQL en lugar de suponer que salió bien. Las contraseñas que falten se agregan solas al `.env`, y si el backend usa las bases de esa PC, sus usuarios quedan escritos en `backend\.env`.

`.\panel-comandos\servicios.ps1` prende el backend, el frontend y el panel de comandos sin abrir ventanas. Como una laptop de 16 GB no aguanta las seis bases y el resto a la vez, el

equipo trabajó casi todo octubre con las bases en otra PC. Esa PC se prepara una vez con `.\preparar-pc-bases.ps1`, que instala lo necesario, abre los puertos solo para Tailscale y corre el arranque, y la PC de trabajo apunta a ella con `.\usar-bases-remotas.ps1`, que cambia direcciones, usuarios y contraseñas del backend, y vuelve con `-Local`. La guía completa está en `docs/instalacion.md` y el modo remoto en `remoto/README.md`.

4.4. Qué pasa si solo quedan una, dos o tres bases

El profesor preguntó qué hace la tienda si no están las seis bases. La prueba de caídas apaga una base por vez, así que el 4 de noviembre se midió también con varias apagadas a la vez, contra las bases de la otra PC. `pruebas/caidas_varias.mjs` apaga con el interruptor todas menos las del escenario, recorre lo que hace una persona en la tienda y compra una unidad, que después cancela por la API.

Tabla 16. Lo que hace la tienda con varias bases apagadas a la vez, medido el 4 de noviembre de 2026.

Bases prendidas	Lo que funciona	Lo que avisa que no puede
Solo PostgreSQL	Catálogo, ficha, búsqueda con filtros y con errores de tecleo, ofertas con descuento, productos parecidos, quienes lo compraron también llevaron, para ti, iniciar sesión, reportes y comprar	Lo más vendido, lo más visto, se compran juntos y tendencias
PostgreSQL y Redis	Todo lo anterior, más lo más vendido, lo más visto y se compran juntos	Tendencias
PostgreSQL y MongoDB	Lo mismo que solo PostgreSQL, y la ficha con sus características	Lo más vendido, lo más visto, se compran juntos y tendencias
PostgreSQL, MongoDB y Redis	Toda la tienda	Tendencias
PostgreSQL, MongoDB y Elasticsearch	Lo mismo que PostgreSQL y MongoDB, con el buscador de siempre	Lo más vendido, lo más visto, se compran juntos y tendencias
Las cinco sin PostgreSQL	Catálogo, ficha, búsqueda, los estantes de la portada, para ti y la sesión ya abierta	Comprar, iniciar sesión, productos parecidos y reportes

Lo explica la misma regla de la sección 4.2 y del registro ADR-22. PostgreSQL guarda el dinero, el stock y los pedidos, así que mientras esté prendido se puede comprar, sea cual sea el resto. Los rankings de la portada salen de Redis, y si Redis no está se recalculan desde ClickHouse. Con los dos apagados no hay de dónde sacarlos y la portada los esconde sin enseñar ningún error. Las tendencias del administrador solo viven en ClickHouse. Sin PostgreSQL la tienda se puede mirar pero no comprar, y lo dice con un aviso en lugar de quedarse esperando.

En los seis casos, al prender otra vez las bases, el backend volvió a usarlas solo, sin reiniciar nada, y ningún paso dio error ni se quedó sin contestar. Con MongoDB apagado la primera

página del catálogo tardó unos 3 segundos, el límite de su cliente, y después el cortacircuitos la contestó al instante. El detalle está en `docs/QUE-PASA-SI-SE-CAE-UNA-BASE.md`.

5. REGISTRO DE DECISIONES DEL SEMESTRE

5.1. Un solo registro

Cada entrega escribió sus decisiones con su propia numeración, y los números terminaron chocando. En `docs/base-de-datos.md` el ADR-3 es el de las imágenes, en el informe de la Entrega 2 el ADR-3 es grafos contra columnar, y la retrospectiva de las versiones anteriores de este informe citaba el segundo. Además, varias decisiones de octubre se implementaron sin escribir su registro, que es lo que el profesor señaló el 28 de septiembre cuando dijo que las decisiones eran genéricas.

Desde esta versión hay un solo registro, numerado del 1 al 26 en el orden en que se tomaron. La última columna dice cómo se citó cada uno antes, para que se pueda seguir en los informes anteriores. Los registros 15 a 23 son las decisiones que faltaban y se escriben completos en esta sección, con el flujo que tocan, las bases que leen y escriben y qué pasa si cada una se cae.

Tabla 17. El registro único de decisiones del semestre.

Id	Decisión	Cuándo	Estado hoy	Antes se citó como
ADR-1	El catálogo de productos en MongoDB	Entrega 1	Vigente	ADR-1 en `docs/base-de-datos.md`
ADR-2	Las ofertas, el stock y el dinero se quedan en PostgreSQL	Entrega 1	Vigente	ADR-2 en `docs/base-de-datos.md`
ADR-3	Las imágenes, embebidas en su variante	Entrega 1	Vigente	ADR-3 en `docs/base-de-datos.md`
ADR-4	Quién propuso un producto se guarda en PostgreSQL	Entrega 1	Vigente	ADR-4 en `docs/base-de-datos.md`
ADR-5	Reseñas, comentarios y likes se quedan en PostgreSQL	Entrega 2	Vigente	ADR-5 en `docs/base-de-datos.md`
ADR-6	El carrito en Redis con vencimiento	Entrega 2	Revisada por ADR-21	ADR-4 en el informe de la Entrega 2
ADR-7	La oferta limitada con un script Lua en Redis	Entrega 2	Vigente, la reserva también queda en PostgreSQL desde ADR-21	ADR-5 en el informe de la Entrega 2
ADR-8	Grafos contra columnar, decisión del 3 de septiembre	Entrega 2	Reemplazada por ADR-10 el 6 de septiembre	ADR-3 en el informe de la Entrega 2
ADR-9	Neo4j para los anillos de reseñas	Entrega 2	Vigente, ampliada por ADR-19	ADR-7 en el informe de la Entrega 2
ADR-10	ClickHouse para eventos y tendencias	Entrega 2	Vigente	ADR-6 en el informe de la Entrega 2
ADR-11	Elasticsearch para la búsqueda del catálogo	Entrega 3	Vigente, ampliada por ADR-20 y ADR-23	ADR-9 en el informe de la Entrega 3

Id	Decisión	Cuándo	Estado hoy	Antes se citó como
ADR-12	Los pagos se quedan en PostgreSQL y no van a NewSQL	Entrega 3	Vigente	ADR-8 en el informe de la Entrega 3
ADR-13	Los vectores en pgvector	Entrega final	Vigente desde el 16 de octubre, tras dos revisiones	ADR-10 en las versiones anteriores de este informe
ADR-14	Control de acceso por servicio en cada base	Entrega final	Ampliada el 23 de octubre	ADR-11 en las versiones anteriores de este informe
ADR-15	El registro de inicios de sesión en MongoDB con copia en Elasticsearch	10 de septiembre	Vigente	No se había registrado
ADR-16	El vencimiento de los descuentos en PostgreSQL	26 de septiembre	Vigente	No se había registrado
ADR-17	Un solo modelo de señales de fraude	30 de septiembre	Vigente	Propuesta ADR-14 en `docs/PROPUESTAS-FRAUDE-Y-DEPENDENCIAS.md`
ADR-18	Comprar sin MongoDB	1 de octubre	Vigente	Propuesta ADR-12 en el mismo documento
ADR-19	Las compras en el grafo de Neo4j	16 de octubre	Vigente	No se había registrado
ADR-20	El buscador de ofertas y los avisos con percolator	17 de octubre	Vigente	No se había registrado
ADR-21	La verdad del carrito y de las reservas en PostgreSQL	17 de octubre	Vigente	Propuesta ADR-13 en el mismo documento
ADR-22	Un respaldo por base, con cortacircuitos	20 de octubre	Vigente	No se había registrado
ADR-23	La tarjeta del buscador sale entera de Elasticsearch	23 de octubre	Vigente	No se había registrado
ADR-24	Un ACL en la aplicación, por oferta	26 de octubre	Vigente	Nueva
ADR-25	La categoría del producto también en el grafo	26 de octubre	Vigente	Nueva
ADR-26	Un ACL de funcionalidades, por rol, en PostgreSQL	28 de octubre	Vigente	Nueva

De las seis decisiones que propusimos después de la revisión del 28 de septiembre, tres se hicieron tal cual, ADR-17, ADR-18 y ADR-21. Las detecciones como serie de tiempo en ClickHouse y el grafo de identidad para armar los casos no se implementaron, y la búsqueda híbrida terminó con el vector solo en pgvector, como cuenta ADR-13.

5.2. Las decisiones de septiembre que faltaban

Tabla 18. Registro de decisión ADR-15

ADR-15 El registro de inicios de sesión en MongoDB con copia en Elasticsearch	
Contexto	Cada intento de entrar, bueno o malo, tiene que quedar registrado para detectar fuerza bruta y tomas de cuenta. Hasta el 10 de septiembre solo se guardaban los inicios buenos, en `log_login` de PostgreSQL, con una copia en ClickHouse. Un ataque de fuerza bruta son cientos de intentos por minuto y cada uno trae datos del cliente que cambian de un navegador a otro.
Alternativas consideradas	1. Seguir en PostgreSQL y agregar los fallos. 2. Escribirlos directo en ClickHouse. 3. Una colección de MongoDB con una copia en Elasticsearch para buscarlos.
Decisión	La colección `eventos_autenticacion` de MongoDB, escrita por el backend sin esperar, y un worker que copia cada evento al índice `mesh_eventos_autenticacion` de Elasticsearch. Tres contadores de Redis vigilan los intentos de la última ventana y dejan la señal en PostgreSQL.
Justificación	**Patrón de consulta.** El administrador busca por cuenta, por IP o por lo que se tecleó, que es búsqueda de texto y filtros, y para eso está Elasticsearch y Kibana. ClickHouse agrega bien pero no edita filas, y la copia necesita marcar cada evento como copiado. **Escalabilidad.** Es la colección que más se escribe del proyecto, un documento por intento, y MongoDB acepta los campos del cliente sin cambiar un esquema. **Consistencia.** El inicio de sesión no espera a MongoDB. Si MongoDB está caído, la persona entra igual y ese evento se pierde, porque esto es un registro y no una fuente de verdad. Lo que sí importa, la señal de fraude, queda en PostgreSQL.
Consecuencias	El inicio de sesión solo depende de PostgreSQL. Aceptamos perder los eventos del rato en que MongoDB está caído, que `log_login` y su copia a ClickHouse quedaran congeladas y una base más en el camino de la detección de fraude.

Tabla 19. Registro de decisión ADR-16

ADR-16 El vencimiento de los descuentos y de las ventas relámpago en PostgreSQL	
Contexto	Un descuento o una venta relámpago pueden vencer solos a una hora. Hasta el 26 de septiembre ese vencimiento era una clave con tiempo de vida en Redis, y el worker que apaga descuentos daba por vencido todo descuento cuya clave no existiera. Redis no distingue una clave que venció de una que se perdió, y los 13 descuentos de la semilla se borraban en su primer minuto.
Alternativas consideradas	1. Seguir con la clave en Redis. 2. Una persistencia de Redis más estricta. 3. Dos columnas de fecha en `oferta` de PostgreSQL.
Decisión	Las columnas `fecha_vencimiento_descuento` y `fecha_vencimiento_venta_limitada` en `oferta`, migración 028, donde vacío quiere decir permanente. El worker lee PostgreSQL cada minuto.
Justificación	**Consistencia.** El descuento cambia el precio que se cobra, así que es dinero y vive donde vive el dinero, en la misma transacción que la oferta. **Costo operativo.** Dos columnas y una migración. La persistencia más estricta de Redis no arreglaba un Redis vaciado a mano ni un volumen borrado.
Consecuencias	Un Redis vaciado ya no borra descuentos. Aceptamos que el worker consulte PostgreSQL cada minuto. Si Redis se vaciaba a media venta relámpago, el contador quedaba en pausa hasta que la venta venciera, y eso se resolvió el 17 de octubre con las reservas en PostgreSQL (ADR-21).

Tabla 20. Registro de decisión ADR-17

ADR-17 Un solo modelo de señales de fraude	
Contexto	Cada familia de detección guardaba su resultado a su manera, el inicio de sesión en `alerta_fraude`, Neo4j en `anillo_fraude` y ClickHouse en `hallazgo_evento_fraude`. En la revisión de la Entrega 3 el profesor señaló que así un caso no podía juntar la evidencia de varias fuentes.
Alternativas consideradas	1. Mantener una tabla por familia y unirlas en una vista. 2. Un documento por caso en MongoDB. 3. Una tabla `senal_fraude` en PostgreSQL con tipo, fuente y evidencia.
Decisión	La tabla `senal_fraude`, migraciones 031 a 033, con un catálogo de tipos y de fuentes. Las señales se agrupan en casos que el administrador confirma o descarta, y los avisos al usuario cuelgan de la señal.
Justificación	**Consistencia.** Casos, evidencias y avisos apuntan a la señal con llave foránea, y las migraciones copiaron las tablas viejas una sola vez. **Patrón de consulta.** El administrador revisa por caso y por cuenta, que en una sola tabla es un JOIN. La vista de la alternativa 1 tenía que repetir la forma de cada familia. **Escalabilidad.** Las señales por el texto de las reseñas de octubre entraron como tipos nuevos, sin tabla nueva.
Consecuencias	Neo4j, ClickHouse, Elasticsearch, PostgreSQL y los contadores del inicio de sesión escriben lo mismo. Aceptamos que todos dependen de PostgreSQL para dejar su resultado y que las tres tablas viejas quedaron sin escrituras.

5.3. Comprar sin MongoDB

Tabla 21. Registro de decisión ADR-18

ADR-18 Comprar sin MongoDB	
Contexto	La prueba de falla del checkout encontró que con MongoDB caído no se podía comprar. El checkout leía de MongoDB el nombre del producto y el nombre y el estado de la variante, aunque MongoDB no está en la transacción, y después de cobrar esperaba a recalcular el catálogo en MongoDB.
Alternativas consideradas	1. No cambiar y avisar que no se puede comprar. 2. Copiar el nombre en columnas de `oferta`. 3. Dos tablas de copia del catálogo en PostgreSQL y una cola para recalcular MongoDB.
Decisión	La alternativa 3. `producto_catalogo` y `variante_catalogo` guardan lo que la compra lee, migración 029, y la cola `evento_producto`, migración 030, lleva cada cambio a un worker que recalcula MongoDB cada 5 segundos. La migración 047 sumó lo que hace falta para mostrar la ficha y el catálogo sin MongoDB.
Justificación	**Consistencia.** Comprar ahora lee y escribe solo PostgreSQL, y Redis como copia rápida. Lo que se cobra y lo que se lee para cobrar están en la misma base. **Patrón de consulta.** El nombre es de la variante y no de la oferta. Con la alternativa 2 se repetía en cada oferta y un cambio de nombre tocaba muchas filas. **Escalabilidad.** La cola guarda una fila pendiente por producto, así que 50 cambios seguidos se resuelven con un solo recálculo.
Consecuencias	Se compra con MongoDB caído, medido con `bash pruebas/falla_checkout.sh`, que lo apaga y da 32 en verde. Aceptamos una copia que el backend y el arranque mantienen al día, que la tarjeta del catálogo tarde hasta 5 segundos en reflejar un cambio de precio y que las características del producto solo existan en MongoDB.

5.4. Las decisiones de octubre

Tabla 22. Registro de decisión ADR-19

ADR-19 Las compras en el grafo de Neo4j	
Contexto	Las recomendaciones para ti y quienes lo compraron también llevaron se calculaban con un worker que guardaba listas ya hechas, y en la revisión del 16 de octubre se pidió leerlas del grafo. El grafo ya tenía cuentas, reseñas y productos.
Alternativas consideradas	1. Seguir con SQL sobre los pedidos y listas calculadas. 2. Llevar las compras al grafo y leer las recomendaciones en vivo.
Decisión	Un nodo `Compra` por línea de pedido, con aristas a la cuenta, al producto y al vendedor, migración 041. Una cola por cuenta, `evento_grafo_compra`, que un worker vacía cada 10 segundos. Las dos rutas leen del grafo en vivo y el worker de listas se borró.
Justificación	**Patrón de consulta.** Quienes compraron esto también compraron aquello es un recorrido de dos o tres saltos, que en Cypher es una línea. **Consistencia.** El pedido se confirma sin esperar a Neo4j, la cola va en la misma transacción que el pedido, y el worker relee todas las compras de la cuenta, así que el grafo queda igual a PostgreSQL. **Costo operativo.** Es la misma cola y el mismo worker que ya llevaban las reseñas al grafo.
Consecuencias	Las compras y los anillos de reseñas están en el mismo grafo y se pueden cruzar. Si Neo4j se cae, las dos recomendaciones salen de los pedidos en PostgreSQL (ADR-22). Aceptamos que el grafo vaya hasta 10 segundos atrás.

Tabla 23. Registro de decisión ADR-20

ADR-20 El buscador de ofertas y los avisos con percolator	
Contexto	El vendedor necesitaba buscar entre sus ofertas por texto, estado y precio, y el comprador quería un aviso cuando llegara algo que buscó sin encontrar o cuando bajara un precio. En PostgreSQL eso es comparar cada oferta nueva contra todos los avisos pendientes.
Alternativas consideradas	1. Consultas en PostgreSQL y un worker que compare cada oferta contra todos los avisos. 2. Un índice de ofertas en Elasticsearch y los avisos como consultas guardadas en un percolator.
Decisión	El índice `mesh_ofertas`, alimentado por la cola `evento_oferta`, migración 043, y el percolator `mesh_seguimientos`, migración 044. Cada oferta que entra al índice pasa por el percolator y el primer aviso que coincide se cumple una sola vez.
Justificación	**Patrón de consulta.** El percolator invierte la búsqueda, guarda las consultas y pregunta cuáles coinciden con un documento nuevo, que es justo lo que pide un aviso. **Consistencia.** El aviso vive en PostgreSQL y en Elasticsearch solo su consulta. Si el índice se rehace, el arranque vuelve a cargar las consultas de los avisos vivos.
Consecuencias	Si Elasticsearch se cae, la cola espera y los avisos llegan cuando vuelve. Aceptamos dos índices más que el arranque prepara y una cola más que vigilar.

Tabla 24. Registro de decisión ADR-21

ADR-21 La verdad del carrito y de las reservas en PostgreSQL	
Contexto	El profesor pidió que Redis nunca fuera la fuente de verdad. Hasta el 17 de octubre el carrito vivía solo en Redis (ADR-6) y la reserva de una venta relámpago solo en el contador de Redis, así que un Redis vaciado se llevaba carritos y reservas sin dejar rastro.
Alternativas consideradas	1. Seguir en Redis con la persistencia activada. 2. Volver a PostgreSQL y sacar a Redis del carrito. 3. Escribir primero en Redis y después en PostgreSQL, que manda.
Decisión	La alternativa 3, migración 045. El carrito es el pedido `pendiente` de la cuenta, uno por cuenta, con su versión y su última actividad, y cada reserva de venta relámpago queda en `reserva_venta_relampago`. Redis decide rápido con su script y PostgreSQL guarda.
Justificación	**Consistencia.** Lo que no se puede reconstruir vive en la base con transacciones. La versión del pedido dice si Redis se quedó atrás, y las reservas en PostgreSQL rehacen el contador de Redis si se pierde. **Patrón de consulta.** Las lecturas del carrito siguen saliendo de Redis, que es lo que se pide en cada página. **Costo operativo.** La alternativa 2 era más simple, pero sumaba una escritura transaccional a cada clic del carrito y perdía el contador atómico de la venta relámpago.
Consecuencias	Un Redis vaciado ya no pierde carritos ni reservas, y si Redis se cae el carrito sale de PostgreSQL. Aceptamos escribir dos veces y un momento en que Redis va adelante de PostgreSQL. Esta decisión revisa ADR-6 y deja falsa la justificación original de ADR-14, que se corrigió.

Tabla 25. Registro de decisión ADR-22

ADR-22 Un respaldo por base, con cortacircuitos	
Contexto	El profesor pidió que si se apaga una base la tienda siga respondiendo con las que quedan, y que se vea en vivo. Al medirlo, con MongoDB caído el catálogo daba error, con Elasticsearch caído el buscador también, y con Redis congelado cada sección de la portada esperaba 10 segundos.
Alternativas consideradas	1. Avisar con un error por sección. 2. Réplicas de cada base. 3. Un respaldo por base en las otras, con un cortacircuitos que tras una caída deja de preguntarle 15 segundos.
Decisión	La alternativa 3, migraciones 047 y 048. Cada base tiene de dónde sacar lo que daba, casi siempre de PostgreSQL, y cada cliente del backend tiene su cortacircuitos.
Justificación	**Consistencia.** PostgreSQL ya tenía la verdad o la copia de casi todo, así que el respaldo lee datos que ya existen. **Costo operativo.** Las réplicas doblaban los contenedores en una laptop de 16 GB que ya no aguantaba las seis bases. **Escalabilidad.** El cortacircuitos evita que cada petición espere el tiempo límite de una base caída, que eran 3 segundos en MongoDB y 8 en Elasticsearch.
Consecuencias	La tienda sigue mostrando lo mismo con cualquier base caída menos PostgreSQL, a veces con menos detalle, medido con `node pruebas/caidas.mjs` en 51 comprobaciones con las bases apagadas y otras 51 congeladas. Aceptamos perder los eventos de ClickHouse y los contadores de Redis de ese rato y mantener dos caminos de código por cada fuente.

Tabla 26. Registro de decisión ADR-23

ADR-23 La tarjeta del buscador sale entera de Elasticsearch	
Contexto	El profesor pidió que el resultado de una búsqueda traiga la foto del producto y salga todo de una vez desde Elasticsearch. Hasta el 23 de octubre la tarjeta del buscador no tenía foto, y conseguirla obligaba a una segunda consulta por resultado.
Alternativas consideradas	1. Dejar la tarjeta sin foto. 2. Pedir la foto a PostgreSQL después de buscar. 3. Guardar el enlace de la foto en el documento del índice, sin indexarlo.
Decisión	La alternativa 3. El campo `imagen` del índice `mesh_productos` guarda el enlace de la primera foto de la primera variante y no se indexa. Lo llenan el ETL y el worker del buscador, lo devuelven las seis formas de buscar, y el respaldo de PostgreSQL lo saca de `variante_catalogo`.
Justificación	**Patrón de consulta.** Una sola consulta pinta la tarjeta completa, como ya pasaba con el precio y el stock, que viajan en el mismo documento. **Costo operativo.** Un campo que no se indexa no agrega trabajo a la búsqueda ni memoria a las agregaciones, solo el enlace guardado.
Consecuencias	El buscador enseña la foto sin preguntarle a otra base, y con Elasticsearch caído la saca de la copia en PostgreSQL. Aceptamos un índice un poco más grande y que el arranque lo rehaga si le falta el campo.

Tabla 27. Registro de decisión ADR-24

ADR-24 Un ACL en la aplicación, por oferta	
Contexto	El control de acceso de las bases es por servicio. El backend entra como `mesh_backend` y ninguna base sabe qué persona está detrás de una petición. Dentro de la tienda la regla era fija y estaba escrita en el SQL, una oferta la toca solo su dueño, y no había forma de que la tienda dejara a un empleado cambiar un precio sin darle su contraseña. El profesor comentó que el ACL podía vivir en la aplicación.
Alternativas consideradas	1. No cambiar nada, con los roles y la regla del dueño. 2. Usuarios de base por persona y políticas por fila en PostgreSQL. 3. Reescribir cada condición del dueño como mío o compartido conmigo, unos veinte lugares. 4. Un ACL en tablas de PostgreSQL, revisado por un guard del backend, en rutas nuevas para lo compartido.
Decisión	La alternativa 4, con la migración 049. `permiso_oferta` guarda la oferta, la cuenta, el permiso (ver, editar o descuento), quién lo dio y hasta cuándo, y `auditoria_permiso_oferta` guarda el registro. `PermisoOfertaGuard` revisa cada ruta de `/compartidas/ofertas`. Sin ninguna entrada vigente contesta 404, el mismo de una oferta ajena, y con alguna pero no la que pide la ruta contesta 403. El dueño reparte en `/vendedor/ofertas/:id/permisos`.
Justificación	**Consistencia.** Las rutas nuevas llaman al mismo servicio de ofertas a nombre del dueño, así que las reglas de precio, descuento y existencia no se copian, y las rutas de lo propio no cambian. **Patrón de consulta.** Es un ACL clásico, una lista pegada a cada oferta que niega lo que no está, y vive donde se conoce a la persona.
Consecuencias	Cada permiso se revisa por separado, vence solo y se quita al instante, y cada cambio queda en el registro con quién lo hizo de verdad. Se prueba con `pruebas/acl_ofertas.sh`, 85 comprobaciones, y con miles de acciones al azar contra un modelo, ninguna mal. Aceptamos que el registro de una acción se escriba después de la acción y no en su misma transacción. No reemplaza el control de acceso de las bases que pide el enunciado, lo complementa con la capa de las personas.

Tabla 28. Registro de decisión ADR-25

ADR-25 La categoría del producto también en el grafo	
Contexto	Quienes lo compraron también llevaron ordenaba solo por cuántas cuentas compraron los dos productos, y en la ficha de una laptop salía primero un teléfono con cuatro compradores que otra laptop con uno. Se pidió que los de la misma categoría vayan siempre primero. El nodo del producto en Neo4j solo tenía su id y su nombre.
Alternativas consideradas	1. No cambiar nada. 2. Reordenar en el backend después de la consulta. 3. Guardar en el grafo toda la ficha del producto. 4. Guardar solo la categoría en el nodo y ordenar en el Cypher.
Decisión	La alternativa 4. Cada producto del grafo lleva su categoría, que sale de `producto_catalogo`. La ponen los workers de compras y de reseñas, el ETL y un worker que cada minuto la compara con PostgreSQL. El Cypher pone primero el grupo de la misma categoría, y el respaldo de PostgreSQL ordena igual.
Justificación	**Patrón de consulta.** El orden se resuelve en la misma consulta de la recomendación, antes del límite. **Costo operativo.** La categoría casi no cambia y copiarla cuesta poco. El precio y la existencia cambian a cada rato y no se copian.
Consecuencias	Con Neo4j y con MongoDB caídos, los 100 productos comprados dan la misma lista en el mismo orden. Un cambio de categoría llega al grafo en menos de un minuto, y mientras tanto el orden puede usar la categoría vieja. Desde el 5 de noviembre los parecidos por el vector siguen el mismo orden, primero su categoría (sección 2.4).

Tabla 29. Registro de decisión ADR-26

ADR-26 Un ACL de funcionalidades, por rol, en PostgreSQL	
Contexto	El profesor pidió el control de acceso en dos niveles, el de las bases y el de las funcionalidades, y que se pueda enseñar llamando al backend directo con una cuenta sin permiso. El de las bases ya estaba (ADR-14). El de las funciones existía como roles escritos en 38 lugares del código, que daban 403 al rol equivocado, pero la lista no se podía enseñar como datos ni cambiar sin compilar.
Alternativas consideradas	1. No cambiar nada y explicar los roles del código como el control de las funciones. 2. Una tabla de permisos por persona para cada función. 3. Usuarios de PostgreSQL por persona con Row Level Security. 4. Las acciones y quién las tiene en tablas de PostgreSQL, por rol, revisadas en cada petición por un guard del backend.
Decisión	La alternativa 4, con la migración 050, en tercera forma normal. `modulo_permiso` y `permiso` son los catálogos, 17 módulos y 38 acciones. `rol_permiso` es el ACL, una fila por rol y acción, y `auditoria_permiso` guarda cada intento negado y cada cambio. Cada ruta privada declara su acción y `PermisoGuard` pregunta a la tabla en cada petición. Sin el permiso, 403.
Justificación	**Consistencia.** La semilla deja pasar exactamente a los mismos roles que antes, ruta por ruta, comprobado contra la matriz guardada antes del cambio. **Costo operativo.** Cambiar lo que puede un rol es una fila, sin compilar ni reiniciar, y una ruta privada sin su acción declarada también se niega.
Consecuencias	Cada petición privada hace una consulta más, por llave. Si PostgreSQL no contesta, el guard usa la última matriz que leyó, igual que la sesión se cree a su token. Los permisos por persona siguen siendo los de las ofertas (ADR-24), y una ruta compartida pasa por los dos.

6. FRAUDE EN RESEÑAS Y CLASIFICADOR

6.1. Un solo modelo de señales

En la revisión de la Entrega 3 el profesor señaló que cada familia de detección guardaba su resultado a su manera. Desde el 30 de septiembre todas guardan lo mismo, una señal en `senal_fraude` con su tipo, su fuente, su evidencia y cuántas veces se volvió a detectar. Las señales se agrupan en casos que el administrador revisa, confirma o descarta. Las migraciones 031 a 033 copiaron las tablas anteriores una sola vez, y su registro es ADR-17.

Con un vector por reseña salieron señales que antes no se podían calcular. Desde el 16 de octubre los pares de reseñas casi iguales los encuentra PostgreSQL con pgvector, sin sacar los vectores de la base, y las notas que no cuadran son una agregación en SQL.

Tabla 30. Las señales nuevas por el texto de las reseñas.

Señal	Qué detecta	Fuente	Migración
Reseñas repetidas	Una cuenta con 5 o más reseñas casi iguales, coseno de 0,92 o más	PostgreSQL con pgvector	035
Anillo de texto	Tres o más cuentas con reseñas casi iguales sobre el mismo vendedor en 30 días	PostgreSQL con pgvector	035
Notas incoherentes por cuenta, producto y vendedor	Texto malo con 4 o 5 estrellas o bueno con 1 o 2, según el clasificador, 5 o más	PostgreSQL con pgvector	037
Reseñas parecidas por producto y por vendedor	5 o más reseñas con una gemela de otra cuenta en 30 días	PostgreSQL con pgvector	038
Reseña positiva con compra cancelada	Una reseña de 4 o 5 estrellas y la compra cancelada dentro de 72 horas	PostgreSQL	039
Reseñas negativas canceladas por vendedor	5 o más reseñas de 1 o 2 estrellas contra un vendedor con la compra cancelada, que sale como víctima	PostgreSQL	039

La demo `.\demo-resenas-texto.ps1` provoca cada una por separado, y cada señal aparece sola a los tres o cuatro segundos, porque se revisan al momento y no solo cada 15 minutos. Como control sembramos 480 reseñas escritas a mano en 40 cuentas con su compra pagada, y ninguna de esas cuentas sale en una señal.

6.2. El clasificador de reseñas

La señal de nota incoherente necesita saber si el texto de una reseña es bueno o malo sin mirar las estrellas. Para eso entrenamos dos modelos sobre el mismo vector de 768 números, una regresión logística y una red neuronal pequeña de 768 entradas, 32 neuronas y una salida. Los dos están escritos a mano con numpy, y la regresión da lo mismo que la de scikit-learn.

Tabla 31. Medición del 2 de octubre, en clasificador-resenas.

Conjunto	Reseñas	Resultado
Entrenamiento y prueba	200 buenas y 200 malas escritas a mano	Los dos aciertan el 100 por ciento de la prueba
Casos difíciles que nunca entrenan	80, con sarcasmo, sin tildes y expresiones locales	La red acierta 91 por ciento y la regresión 90
Sarcasmo	3	Los dos fallan las tres

Por el sarcasmo y por los casos difíciles, la señal nunca bloquea sola. Solo se marca cuando la probabilidad de texto malo pasa de 0,8 con cuatro o cinco estrellas, o baja de 0,2 con una o dos, y va al administrador para que la revise.

7. RETROSPECTIVA

7.1. Cómo la hicimos

Para cada decisión miramos qué prometía su registro y qué pasó después, con lo medido en las pruebas y lo que tuvimos que arreglar. Una decisión se sostiene si lo que prometió se cumplió y lo que aceptó como costo no nos cobró más de la cuenta. Se replantea si tuvimos que rodearla o si hoy la tomaríamos distinto con lo que sabemos.

Separamos lo técnico de lo que aprendimos sobre la forma de trabajar, porque varias de las cosas que replantearíamos tienen que ver con el orden en que hicimos el trabajo.

7.2. Lo que sostendríamos

Tabla 32. Las decisiones que tomaríamos igual.

Decisión	Por qué se sostiene	Evidencia
El dinero y el stock en PostgreSQL, sin migrar las ofertas ni los pagos (ADR-2 y ADR-12)	Es la única pieza donde un error cuesta dinero, y una transacción local es fácil de probar	Cero sobreventas y cero descuadres sobre 555 ofertas. La prueba de falla del checkout dio 32 en verde y ninguna caída cobra a medias
La oferta limitada con un script Lua en Redis (ADR-7)	Decide quién gana cada unidad sin bloqueos en la aplicación	Cero sobreventas con 5000 solicitudes y 200 clientes simultáneos
El catálogo en MongoDB (ADR-1)	Cada categoría trae atributos distintos y ninguna tabla fija los aguanta sin columnas vacías	El llenado promedio de atributos es 59 por ciento, y se filtra por atributo sin tocar el esquema
Neo4j para los anillos de reseñas, alimentado con una cola transaccional (ADR-9)	La consulta de varios saltos se escribe una vez y se lee sin esfuerzo	Desde la migración 015 el grafo se mantiene solo y una ruta de conciliación compara sus conteos con PostgreSQL
Elasticsearch como copia reconstruible del catálogo (ADR-11)	No guarda nada propio, así que perderlo no pierde datos	Se reconstruye entero en el arranque y la prueba de facetas dio 26 en verde
Calcular los vectores con Ollama en la misma máquina (ADR-13)	Ningún texto sale a un servicio externo y no hay costo por consulta	1003 vectores, 1000 de productos activos, y el worker del backend da el mismo vector que el ETL, con distancia cero
Un solo comando de arranque que cuenta lo que quedó	Una PC nueva queda lista sin pasos a mano y el script no dice listo sin haberlo medido	El arranque compara cada base contra PostgreSQL y avisa en rojo si algo no cuadra

La que más pesó fue dejar el dinero en PostgreSQL. Cada vez que una base nueva entró al sistema, la pregunta fue qué pasa con el checkout si esa base se cae, y la respuesta fue siempre la misma, el pedido se cobra en una sola transacción y todo lo demás se puede reconstruir o se

avisa. La evaluación de NewSQL de la entrega anterior llegó a la misma conclusión con otro camino.

7.3. Lo que replantearíamos

Tabla 33. Las decisiones que tomaríamos distinto.

Decisión	Qué pasó	Qué haríamos hoy
Dos repartidores de identificador de producto, la secuencia de PostgreSQL y el mayor identificador de MongoDB	No se hablan. Medimos una secuencia adelantada 21 números respecto al máximo real, y el arranque tuvo que ganar una revisión que avisa si los dos se cruzan	Un solo dueño del identificador desde la Entrega 1, la secuencia de PostgreSQL, y MongoDB recibiéndolo
El vencimiento de un descuento guardado solo como clave con tiempo de vida en Redis (corregido en ADR-16)	El proceso que quita descuentos vencidos no distingue una clave que venció de una que nunca existió. Los 13 descuentos de la semilla se borraban en su primer minuto y un Redis vaciado borraría todos. El 26 de septiembre el vencimiento pasó a una columna de PostgreSQL	El vencimiento en PostgreSQL desde el principio y Redis solo como acelerador. En Redis va lo que se puede perder
El carrito y las reservas de la venta relámpago solo en Redis (ADR-6, corregido en ADR-21)	Un Redis vaciado se llevaba los carritos abiertos y las reservas sin dejar rastro. El 17 de octubre, por pedido del profesor, la verdad pasó al pedido pendiente y a una tabla de reservas en PostgreSQL, y Redis quedó como copia rápida	Redis como acelerador desde el principio. Lo que no se puede reconstruir va en la base con transacciones, aunque eso cueste una escritura más por clic
Aplazar la base columnar en la Entrega 2 (ADR-8)	La aplazamos el 3 de septiembre y la metimos el 6. Decidimos con las consultas que ya teníamos, y el fraude consiste en hacer preguntas que no sabíamos que íbamos a hacer	Decidir por el tipo de preguntas que vienen y no solo por las que ya existen
Escribir los eventos a ClickHouse sin respaldo (ADR-10)	Si ClickHouse está caído, los eventos de ese rato se pierden y los rankings quedan cortos sin avisar	Una cola en PostgreSQL, igual que la del grafo, que se vacía cuando ClickHouse vuelve
El registro de inicios de sesión primero en ClickHouse (corregido en ADR-15)	Terminó en MongoDB con copia en Elasticsearch, y la copia a ClickHouse quedó congelada	Elegir el destino por quién lo va a consultar antes de escribir el primer evento
El checkout leyendo de MongoDB sin que MongoDB esté en la transacción (corregido en ADR-18)	La prueba de falla encontró que con MongoDB caído no se podía comprar. El 1 de octubre copiamos a PostgreSQL lo que el checkout lee y hoy se compra con MongoDB caído	Que desde la Entrega 1 todo lo que el checkout lee viva en PostgreSQL y MongoDB sea solo para pintar
Mudar el vector a Elasticsearch por lo cómodo de la consulta (primera revisión de ADR-13)	Ganamos una sola consulta para la búsqueda híbrida, pero su índice comprimido solo coincidía con el coseno exacto en 317 de 1001 productos. El 16 de octubre el vector volvió a pgvector, que coincide en 1000	Medir la precisión de cada opción contra el cálculo exacto antes de mudar un dato, y no solo lo cómoda que queda la consulta

Las ocho tienen algo en común. En casi todas una copia terminó comportándose como si fuera la verdad. El identificador de MongoDB, la clave de Redis, el carrito que solo existía en Redis,

el evento que solo existía en ClickHouse. Cuando la copia se pierde o se adelanta, algo que debía reconstruirse se rompe. La mudanza del vector enseñó otra cosa, que una decisión cómoda también hay que medirla. La regla que sacamos es que cada dato tenga un solo dueño escrito, y que todo lo demás se pueda borrar y volver a calcular sin perder nada.

7.4. Lo que aprendimos sobre la forma de trabajar

- Sostendríamos el contrato escrito entre el backend y el resto. Cada pedido llevaba la consulta probada y la forma exacta de la respuesta, y eso permitió trabajar en paralelo sin pisarnos.
- Replantearíamos crear tablas a mano en una base local. Una tabla de reglas de descuento llegó al repositorio sin su migración, solo existía en la base de quien la creó, y hubo que escribirla después a partir del modelo del backend. La migración tiene que viajar en el mismo cambio que el código que la usa.
- Sostendríamos que cada prueba tenga su control. Una negativa sin control sale en verde también con la base caída, y el control es lo que separa un rechazo de verdad de un servicio que no contesta.
- Replantearíamos dejar el control de acceso para el final. Durante casi todo el semestre cualquier servicio pudo vaciar Redis, el backend entró a MongoDB como root y Elasticsearch no pidió contraseña aunque su puerto estaba abierto a la red. Configurarlo desde la Entrega 2 habría costado lo mismo.
- Replantearíamos numerar las decisiones por entrega. Los números chocaron entre los informes y varias decisiones de octubre se implementaron sin su registro. Un solo registro desde el primer día, con el registro escrito en el mismo cambio que el código, lo habría evitado.

8. CONCLUSIONES

8.1. Conclusiones

La recomendación semántica empezó en pgvector, pasó por Elasticsearch para tener el vector junto al texto y volvió a pgvector, donde coincide con el cálculo exacto. En el camino abrió la búsqueda híbrida, la búsqueda por intención y las señales de fraude por el texto de las reseñas, todas con el mismo modelo local.

El control de acceso cerró un riesgo que tuvimos todo el semestre sin notarlo. Hoy el backend entra a cada base con un usuario que solo puede leer y escribir datos, las herramientas de inspección solo miran o solo se abren en la misma PC, Elasticsearch pide contraseña, y la prueba demuestra cada negativa con su control. Neo4j es la excepción, porque su edición gratuita no trae roles. Dentro de la tienda, lo que puede hacer cada rol y cada cuenta sobre una oferta ajena también son listas en PostgreSQL, y un rol sin permiso recibe 403 aunque llame a la ruta directo.

Mirando el semestre completo, lo que mejor funcionó fue decidir por el patrón de consulta y dejar el dinero en una sola transacción. Lo que peor funcionó fueron las copias sin dueño claro. Si empezáramos de nuevo, escribiríamos quién es el dueño de cada dato antes de elegir dónde guardarlo.